

OBJECT ORIENTED PROGRAMMING [CT-260]



NAME: ABDUL RAHEEM SHEIKH

ROLL NO: CT-25192

DEPARTMENT: BCIT

BATCH: 2025

YEAR & SECTION: FSCS-D

LAB: 8

QUESTION#1:

CODE:

```
#include <iostream>
using namespace std;

class Vehicle
{
    string category, brand, paintColor, vehicleModel;
    int manufactureYear, distanceCovered;

public:
    Vehicle(string category, string brand, string paintColor, string vehicleModel,
            int manufactureYear, int distanceCovered)
        : category(category),
          brand(brand),
          paintColor(paintColor),
          vehicleModel(vehicleModel),
          manufactureYear(manufactureYear),
          distanceCovered(distanceCovered) {}

    void displayVehicle()
    {
        cout << "vehicle:" << endl
              << "category: " << category << endl
              << "brand: " << brand << endl
              << "model: " << vehicleModel << endl
              << "color: " << paintColor << endl
              << "manufacturing: " << manufactureYear << endl
              << "distance: " << distanceCovered << endl;
    }
};

class GasVehicle : virtual public Vehicle
{
    float fuelCapacity;

public:
    GasVehicle(string category, string brand, string paintColor, string vehicleModel,
               int manufactureYear, int distanceCovered, float fuelCapacity)
        : Vehicle(category, brand, paintColor, vehicleModel, manufactureYear,
                  distanceCovered),
```

```

        fuelCapacity(fuelCapacity) {}

    void displayGasVehicle()
    {
        cout << "fuel " << fuelCapacity << endl;
    }
};

class ElectricVehicle : virtual public Vehicle
{
    float batteryCapacity;

public:
    ElectricVehicle(string category, string brand, string paintColor, string
vehicleModel,
                    int manufactureYear, int distanceCovered, float batteryCapacity)
        : Vehicle(category, brand, paintColor, vehicleModel, manufactureYear,
distanceCovered),
          batteryCapacity(batteryCapacity) {}

    void displayElectricVehicle()
    {
        cout << "battery " << batteryCapacity << endl;
    }
};

class HighPerformace : public GasVehicle
{
    int horsepower, maxVelocity;

public:
    HighPerformace(string category, string brand, string paintColor, string
vehicleModel,
                    int manufactureYear, int distanceCovered, float fuelCapacity,
                    int horsepower, int maxVelocity)
        : Vehicle(category, brand, paintColor, vehicleModel, manufactureYear,
distanceCovered),
          GasVehicle(category, brand, paintColor, vehicleModel, manufactureYear,
distanceCovered, fuelCapacity),
          horsepower(horsepower),
          maxVelocity(maxVelocity) {}

    void displayHighPerformance()
    {
        displayVehicle();
        displayGasVehicle();

        cout << "Power " << horsepower << endl

```

```

        << "speed " << maxVelocity << endl;
    }
};

class SportsCar : public HighPerformace
{
public:
    string transmissionType, drivetrain;

    SportsCar(string category, string brand, string paintColor, string vehicleModel,
              int manufactureYear, int distanceCovered, float fuelCapacity,
              int horsepower, int maxVelocity,
              string transmissionType, string drivetrain)
        : Vehicle(category, brand, paintColor, vehicleModel, manufactureYear,
distanceCovered),
          HighPerformace(category, brand, paintColor, vehicleModel,
                        manufactureYear, distanceCovered,
                        fuelCapacity, horsepower, maxVelocity),
            transmissionType(transmissionType),
            drivetrain(drivetrain) {}

    void displaySportsCar()
    {
        displayHighPerformance();

        cout << "Gear Box " << transmissionType << endl
              << "Drive Sys " << drivetrain << endl;
    }
};

class HeavyVehicle : public GasVehicle, public ElectricVehicle
{
    int loadCapacity, wheelCount, vehicleLength;

public:
    HeavyVehicle(string category, string brand, string paintColor, string
vehicleModel,
                int manufactureYear, int distanceCovered,
                float fuelCapacity, float batteryCapacity,
                int loadCapacity, int wheelCount, int vehicleLength)
        : Vehicle(category, brand, paintColor, vehicleModel, manufactureYear,
distanceCovered),
          GasVehicle(category, brand, paintColor, vehicleModel, manufactureYear,
distanceCovered, fuelCapacity),
          ElectricVehicle(category, brand, paintColor, vehicleModel, manufactureYear,
distanceCovered, batteryCapacity),
            loadCapacity(loadCapacity),
            wheelCount(wheelCount),

```

```

        vehicleLength(vehicleLength) {}

void displayHeavyVehicle()
{
    displayVehicle();
    displayGasVehicle();
    displayElectricVehicle();

    cout << "weight " << loadCapacity << endl
         << "wheels " << wheelCount << endl
         << "length " << vehicleLength << endl;
}
};

class ConstructionTruck : public HeavyVehicle
{
public:
    string cargoType;

    ConstructionTruck(string category, string brand, string paintColor, string
vehicleModel,
                     int manufactureYear, int distanceCovered,
                     float fuelCapacity, float batteryCapacity,
                     int loadCapacity, int wheelCount,
                     int vehicleLength, string cargoType)
        : Vehicle(category, brand, paintColor, vehicleModel, manufactureYear,
distanceCovered),
          HeavyVehicle(category, brand, paintColor, vehicleModel,
                        manufactureYear, distanceCovered,
                        fuelCapacity, batteryCapacity,
                        loadCapacity, wheelCount, vehicleLength),
            cargoType(cargoType) {}

void displayConstructionTruck()
{
    displayHeavyVehicle();
    cout << "cargo type " << cargoType << endl;
}
};

class Bus : public HeavyVehicle
{
    int seatingCapacity;

public:
    Bus(string category, string brand, string paintColor, string vehicleModel,
        int manufactureYear, int distanceCovered,
        float fuelCapacity, float batteryCapacity,

```

```

        int loadCapacity, int wheelCount,
        int vehicleLength, int seatingCapacity)
        : Vehicle(category, brand, paintColor, vehicleModel, manufactureYear,
distanceCovered),
        HeavyVehicle(category, brand, paintColor, vehicleModel,
            manufactureYear, distanceCovered,
            fuelCapacity, batteryCapacity,
            loadCapacity, wheelCount, vehicleLength),
            seatingCapacity(seatingCapacity) {}

    void displayBus()
    {
        displayHeavyVehicle();
        cout << "seats " << seatingCapacity << endl;
    }
};

int main()
{
    Bus bus("Bus", "Toyota", "White", "Hiace",
        2017, 195000,
        500, 0,
        1100, 6, 40, 30);

    bus.displayBus();

    return 0;
}

```

OUTPUT:

```

vehicle:
category: Bus
brand: Toyota
model: Hiace
color: White
manufacturing: 2017
distance: 195000
fuel 500
battery 0
weight 1100
wheels 6
length 40
seats 30

```

QUESTION#2:

CODE:

```
#include <iostream>
#include <vector>
using namespace std;

class Character
{
protected:
    string characterName;
    int characterLevel, hitPoints;

public:
    Character(string characterName, int characterLevel, int hitPoints)
        : characterName(characterName),
          characterLevel(characterLevel),
          hitPoints(hitPoints) {}

    void showCharacter()
    {
        cout << "Info" << endl
              << "Name = " << characterName << endl
              << "Level = " << characterLevel << endl
              << "Health = " << hitPoints << endl;
    }
};

class Warrior : virtual public Character
{
protected:
    int attackPower, swordSkill;

public:
    Warrior(string characterName, int characterLevel, int hitPoints,
            int attackPower, int swordSkill)
        : Character(characterName, characterLevel, hitPoints),
          attackPower(attackPower),
          swordSkill(swordSkill) {}

    void slash()
    {
        cout << "slashing" << endl;
    }

    void showWarrior()
```

```

    {
        cout << "Strength = " << attackPower << endl
              << "Melee Skill = " << swordSkill << endl;
    }

    void show()
    {
        showCharacter();
        showWarrior();
    }
};

class Mage : virtual public Character
{
protected:
    int magicPower, castingSkill;

public:
    Mage(string characterName, int characterLevel, int hitPoints,
         int magicPower, int castingSkill)
        : Character(characterName, characterLevel, hitPoints),
          magicPower(magicPower),
          castingSkill(castingSkill) {}

    void fireball()
    {
        cout << "fireball" << endl;
    }

    void showMage()
    {
        cout << "Intelligence = " << magicPower << endl
              << "Spell Skill = " << castingSkill << endl;
    }

    void show()
    {
        showCharacter();
        showMage();
    }
};

class Archer : virtual public Character
{
protected:
    int agility, bowSkill;

public:

```



```

    Archer(string characterName, int characterLevel, int hitPoints,
           int agility, int bowSkill)
        : Character(characterName, characterLevel, hitPoints),
          agility(agility),
          bowSkill(bowSkill) {}

    void rapidShot()
    {
        cout << "shooting" << endl;
    }

    void showArcher()
    {
        showCharacter();
        cout << "Dexterity = " << agility << endl
              << "Ranged Skill = " << bowSkill << endl;
    }
};

class NPC : public Character
{
protected:
    string movementPattern, npcDialogue;

public:
    NPC(string characterName, int characterLevel, int hitPoints,
         string npcDialogues, string movementPattern)
        : Character(characterName, characterLevel, hitPoints),
          npcDialogue(npcDialogues),
          movementPattern(movementPattern) {}

    void speak()
    {
        cout << "Dialogue = " << npcDialogue << endl;
    }

    void move()
    {
        cout << movementPattern << endl;
    }

    void showNPC()
    {
        showCharacter();

        cout << "Dialogues=" << endl;
        speak();
    }
};

```

```

        cout << "Moves=" << endl;
        move();
    }
};

class Mighty : public Warrior, public Mage
{
public:
    Mighty(string characterName, int characterLevel, int hitPoints,
           int attackPower, int swordSkill,
           int magicPower, int castingSkill)
        : Character(characterName, characterLevel, hitPoints),
          Warrior(characterName, characterLevel, hitPoints, attackPower, swordSkill),
          Mage(characterName, characterLevel, hitPoints, magicPower, castingSkill) {}

    void spellSlash()
    {
        cout << "Both." << endl;
    }

    void showMighty()
    {
        showCharacter();
        showWarrior();
        showMage();
    }
};

int main()
{
    Warrior warrior("Warrior", 5, 200, 220, 150);
    Mage mage("Mage", 2, 70, 57, 68);
    Archer archer("Archer", 10, 23, 54, 90);
    NPC npc("NPC", 0, 89, "Hi", "Up");
    Mighty mighty("Mighty", 90, 150, 55, 130, 200, 60);

    warrior.show();
    warrior.slash();

    cout << endl;

    mage.show();
    mage.fireball();

    cout << endl;

    archer.showArcher();
    archer.rapidShot();
}

```

```

        cout << endl;

        npc.showNPC();

        cout << endl;

        mighty.showMighty();
        mighty.spellSlash();

        return 0;
}

```

OUTPUT:

```

Info
Name = Warrior
Level = 5
Health = 200
Strength = 220
Melee Skill = 150
slashing

```

```

Info
Name = Mage
Level = 2
Health = 70
Intelligence = 57
Spell Skill = 68
fireball

```

```

Info
Name = Archer
Level = 10
Health = 23
Dexterity = 54
Ranged Skill = 90
shooting

```

```

Info
Name = NPC
Level = 0
Health = 89
Dialogues=
Dialogue = Hi
Moves=
Up

```

Info

Name = Mighty

Level = 90

Health = 150

Strength = 55

Melee Skill = 130

Intelligence = 200

Spell Skill = 60

Both.

QUESTION#3:

CODE:

```
#include <iostream>
using namespace std;

class Account
{
protected:
    double currentBalance;

public:
    Account()
    {
        cout << "Initial Balance = ";
        cin >> currentBalance;
    }

    Account(double currentBalance)
    {
        this->currentBalance = currentBalance;
    }

    virtual void deposit(double depositAmount)
    {
        currentBalance += depositAmount;
        cout << "Amount Deposited = " << depositAmount << endl;
    }

    virtual void withdraw(double withdrawAmount)
    {
        if (withdrawAmount <= currentBalance)
        {
            currentBalance -= withdrawAmount;
            cout << "Amount Withdrawn = " << withdrawAmount << endl;
        }
        else
        {
            cout << "Not enough balance!" << endl;
        }
    }

    void checkBalance()
    {
        cout << "Current Balance = " << currentBalance << endl;
    }
}
```

```

    double getBalance()
    {
        return currentBalance;
    }

    void setBalance(double newBalance)
    {
        currentBalance = newBalance;
    }
};

class InterestAccount : public Account
{
protected:
    double bonusInterest;

public:
    InterestAccount() : Account() {}

    InterestAccount(double amount) : Account(amount) {}

    void deposit(double depositAmount)
    {
        bonusInterest = depositAmount * 0.30;
        currentBalance += depositAmount + bonusInterest;

        cout << "Deposited with 30% interest = "
              << depositAmount + bonusInterest << endl;
    }
};

class ChargingAccount : public Account
{
protected:
    double serviceFee = 25;

public:
    ChargingAccount() : Account() {}

    ChargingAccount(double amount) : Account(amount) {}

    void withdraw(double withdrawAmount)
    {
        double totalDeduction = withdrawAmount + serviceFee;

        if (totalDeduction <= currentBalance)
        {

```

```

        currentBalance -= totalDeduction;

        cout << "Withdrawn = "
              << withdrawAmount
              << " (Fee = Rs.25)" << endl;
    }
    else
    {
        cout << "Insufficient balance including fee!" << endl;
    }
}
};

class ACI
{
public:
    void transfer(double transferAmount, Account &accountObj)
    {
        accountObj.deposit(transferAmount);
        cout << "Transferred = " << transferAmount
              << " to Account" << endl;
    }

    void transfer(double transferAmount, InterestAccount &accountObj)
    {
        accountObj.deposit(transferAmount);
        cout << "Transferred = " << transferAmount
              << " to Interest Account" << endl;
    }

    void transfer(double transferAmount, ChargingAccount &accountObj)
    {
        accountObj.deposit(transferAmount);
        cout << "Transferred = " << transferAmount
              << " to Charging Account" << endl;
    }
};

int main()
{
    cout << "Enter Account 1 Details" << endl;
    Account acc1;

    cout << "Enter Account 2 Details" << endl;
    InterestAccount acc2;

    cout << "Enter Account 3 Details" << endl;
    ChargingAccount acc3;

```

```
    cout << endl;

    acc1.deposit(1000);
    acc2.deposit(1000);

    cout << endl;

    acc1.withdraw(500);
    acc3.withdraw(200);

    cout << endl;

    cout << "Account 1 =" << endl;
    acc1.checkBalance();

    cout << "Account 2 =" << endl;
    acc2.checkBalance();

    cout << "Account 3 =" << endl;
    acc3.checkBalance();

    ACI transactionObj;

    cout << endl;

    transactionObj.transfer(300, acc1);
    transactionObj.transfer(300, acc2);
    transactionObj.transfer(300, acc3);

    cout << endl;

    cout << "Account 1 =" << endl;
    acc1.checkBalance();

    cout << "Account 2 =" << endl;
    acc2.checkBalance();

    cout << "Account 3 =" << endl;
    acc3.checkBalance();

    return 0;
}
```


OUTPUT:

```
Enter Account 1 Details
Initial Balance = 25000
Enter Account 2 Details
Initial Balance = 15000
Enter Account 3 Details
Initial Balance = 30000

Amount Deposited = 1000
Deposited with 30% interest = 1300

Amount Withdrawn = 500
Withdrawn = 200 (Fee = Rs.25)

Account 1 =
Current Balance = 25500
Account 2 =
Current Balance = 16300
Account 3 =
Current Balance = 29775

Amount Deposited = 300
Transferred = 300 to Account
Deposited with 30% interest = 390
Transferred = 300 to Interest Account
Amount Deposited = 300
Transferred = 300 to Charging Account

Account 1 =
Current Balance = 25800
Account 2 =
Current Balance = 16690
Account 3 =
Current Balance = 30075
```

QUESTION#4:

CODE:

```
#include <iostream>
using namespace std;

class Media
{
public:
    virtual void borrow() = 0;
    virtual void returning() = 0;
    virtual void display() = 0;
};

class BookInfo
{
protected:
    string title, writer;
    int totalPages;

public:
    BookInfo(string title, string writer, int totalPages)
        : title(title), writer(writer), totalPages(totalPages) {}
};

class MagazineInfo
{
protected:
    string publisher;
    int edition, issueCode;

public:
    MagazineInfo(string publisher, int issueCode, int edition)
        : publisher(publisher), issueCode(issueCode), edition(edition) {}
};

class DVDInfo
{
protected:
    string movieTitle, producer;
    int totalDiscs;

public:
    DVDInfo(string movieTitle, string producer, int totalDiscs)
        : movieTitle(movieTitle),
          producer(producer),
```

```

        totalDiscs(totalDiscs) {}
};

class Book : protected Media, protected BookInfo
{
public:
    Book(string title, string writer, int totalPages)
        : BookInfo(title, writer, totalPages) {}

    void borrow()
    {
        cout << "Borrowing = " << title << endl;
    }

    void returning()
    {
        cout << "Returning = " << title << endl;
    }

    void display()
    {
        cout << "INFO =" << endl;
        cout << "Title = " << title << endl;
        cout << "Author = " << writer << endl;
        cout << "Pages = " << totalPages << endl;
    }
};

class Magazine : protected Media, protected MagazineInfo
{
public:
    Magazine(string publisher, int issueCode, int edition)
        : MagazineInfo(publisher, issueCode, edition) {}

    void borrow()
    {
        cout << "Borrowing Magazine Issue = "
            << issueCode << endl;
    }

    void returning()
    {
        cout << "Returning Magazine = "
            << issueCode << endl;
    }

    void display()
    {

```

```

        cout << "INFO =" << endl;
        cout << "Issue Number = " << issueCode << endl;
        cout << "Edition = " << edition << endl;
        cout << "Publisher = " << publisher << endl;
    }
};

class DVD : protected Media, protected DVDInfo
{
public:
    DVD(string movieTitle, string producer, int totalDiscs)
        : DVDInfo(movieTitle, producer, totalDiscs) {}

    void borrow()
    {
        cout << "Borrowing = "
             << movieTitle << endl;
    }

    void returning()
    {
        cout << "Returning = " << movieTitle << endl;
    }

    void display()
    {
        cout << "INFO =" << endl;
        cout << "Title = " << movieTitle << endl;
        cout << "Director = " << producer << endl;
        cout << "Discs = " << totalDiscs << endl;
    }
};

int main()
{
    Book b1("Harry", "J.K", 56);
    Magazine m1("Editorial", 123, 456);
    DVD d1("Dvd", "123", 5);
    b1.display();
    m1.display();
    d1.display();

    cout << endl;

    b1.borrow();

    return 0;
}

```

OUTPUT:

```
INFO =  
Title = Harry  
Author = J.K  
Pages = 56  
INFO =  
Issue Number = 123  
Edition = 456  
Publisher = Editorial  
INFO =  
Title = Dvd  
Director = 123  
Discs = 5  
  
Borrowing = Harry
```